

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	KS THARUHESHWAR	Reg. No.:	192572148
Section / Batch:	2025	Date:	06-08-26

Instructions to Students

- Answer ALL debugging and optimisation tasks. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions	Marks
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	



SIMATS Online Programming Lab
DAA PROGRAMMING

V Deepa Dharshini
192525213RD

QUESTIONS

Q1
Brute Force
50 marks

Q2
Brute Force
50 marks

2/2 answered

Q1 Brute Force 50 marks

Debugging Closest Pair Code (Redundant Self-Comparison and Wrong Pairing Logic)

The following brute force Closest Pair implementation compares every point with every other point, including itself, which leads to incorrect minimum distance and redundant computation. Carefully analyze the nested loop structure and explain why self-comparison breaks correctness. Debug the algorithm to compare only valid distinct pairs exactly once. Optimize the implementation by reducing duplicate distance evaluations while preserving brute force behavior. Analyze the time complexity of the corrected solution and rewrite the final code with suitable comments.

```

import math
def closest_pair(points):
    min_dist = float('inf')
    for i in range(len(points)):
        for j in range(len(points)):
            dist = math.sqrt((points[i][0]-points[j][0])**2 +
                             (points[i][1]-points[j][1])**2)
            if dist < min_dist:
                min_dist = dist
    return min_dist

```

Your Code

```

1 import math
2 def closest_pair(points):
3     min_dist = float('inf')
4     for i in range(len(points)):
5         for j in range(i + 1, len(points)):
6             dist = math.sqrt((points[i][0] - points[j][0])**2 +
7                             (points[i][1] - points[j][1])**2)
8             if dist < min_dist:
9                 min_dist = dist
10    return min_dist
11 points = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
12 print("Minimum Distance =", closest_pair(points))

```

Input
stdin input

Output
Minimum Distance =
1.4142135623730951



SIMATS Online Programming Lab
DAA PROGRAMMING

V Deepa Dharshini
192525213RD

QUESTIONS

Q1
Brute Force
50 marks

Q2
Brute Force
50 marks

2/2 answered

Q2 Brute Force 50 marks

Debugging String Matching Code (Incorrect Match Detection Condition)

The following brute force string matching implementation breaks correctly on mismatches, but the condition used to declare a full match is logically unsafe and depends on the final value of the loop variable. Carefully analyze why this may lead to incorrect behavior in some cases. Apply debugging so that a match is reported only when every character in the pattern has been successfully compared. Optimize the implementation for clarity while retaining brute force logic. Analyze the time complexity and rewrite the final corrected code with comments.

```

def string_match(text, pattern):
    n, m = len(text), len(pattern)
    result = []
    for i in range(n - m + 1):
        for j in range(m):
            if text[i+j] != pattern[j]:
                break
        if j == m-1:
            result.append(i)
    return result

```

Your Code

```

1 def string_match(text, pattern):
2     n = len(text)
3     m = len(pattern)
4     result = []
5
6     for i in range(n - m + 1):
7         for j in range(m):
8             if text[i + j] != pattern[j]:
9                 break
10            else:
11                result.append(i)
12    return result
13 text = "ABCDABCBCDA"
14 pattern = "ABCD"
15
16
17 print("Pattern found at positions:\n",string_match(text, patte

```

Input
stdin input

Output
Pattern found at positions:
[0, 7]

Code of Conduct

I V Deepa Dharshini (1925252213) certify that this submission is my original work and that I have adhered to the guidelines specified in the Code of Conduct. Any violation of academic integrity rules will result in disciplinary action.

Signature of the Student

KS THARUNESHWAR

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, wellstructured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging (25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____ Date: _____	Signature: _____ Date: _____	Signature: _____ Date: _____